SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1712**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

*I **Shaik Mohammed Waseem Rayan (192373004)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Shaik Mohammed Waseem Rayan

Annexure A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).
- Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

EXPERIMENT 1: DECISION TREE CLASSIFICATION

Objective

To implement and evaluate a **Decision Tree Classification** model using the Iris dataset, preprocess the data, divide it into training and testing sets, construct the tree using the **Gini Index** as the splitting criterion, and evaluate its performance using accuracy, confusion matrix, classification report, and misclassified instances.

Software / Tools Required

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- NumPy
- Pandas
- Scikit-learn
- Matplotlib

Theory

A **Decision Tree** is a supervised machine learning algorithm used for classification and regression. It recursively divides the dataset into smaller groups using decision rules based on input features.
For classification, the **Gini Index** can be used to measure the impurity of a node.

[

Gini = 1-\sum_{i=1}^{n}p_i^2
]

where (p_i) is the probability of a sample belonging to class (i).

A lower Gini value indicates a purer node. The feature and threshold that provide the best reduction in impurity are selected for splitting.

The Iris dataset contains 150 samples belonging to three classes:

- Iris Setosa
- Iris Versicolor

- Iris Virginica

Each sample has four features:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

(a) Dataset Loading, Pre-processing and Splitting Python Program

```
import pandas as
pd import numpy
as np

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Iris dataset
iris = load_iris(as_frame=True)

X = iris.data
y = iris.target

# Create a complete dataframe for display
df = iris.frame

# Display first 5
rows print("First 5
rows:")
print(df.head())
```

```
# Display data types
print("\nData Types:")
print(X.dtypes)

# Check missing values
print("\nMissing Values:")
print(X.isnull().sum())

# Handle missing values, if present
X = X.fillna(X.median(numeric_only=True))

# Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Output

First 5 rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0

4 5.0 3.6 1.4 0.2 0

Data Types:

sepal length (cm) float64

sepal width (cm) float64

petal length (cm) float64

petal width (cm) float64

Training samples: 120

Testing samples: 30

Pre-processing Analysis

The dataset was checked for missing values before training. No missing values were present. The features were numerical and therefore did not require categorical encoding. The dataset was divided into **80% training data and 20% testing data** using stratified sampling to preserve class proportions.

(b) Decision Tree Construction Using Gini Index

```
# Create Decision Tree classifier
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Display decision tree structure
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
```

)

print("Decision Tree Structure:")

print(tree_rules)

Important Decision Tree Structure

|--- petal length (cm) <= 2.45

| |--- class: 0

|--- petal length (cm) > 2.45

| |--- petal width (cm) <= 1.65

| | |--- petal length (cm) <= 4.95

| | | |--- class: 1

...

The **root node uses Petal Length** because it provides an effective separation between the classes. In particular, a petal length less than or equal to approximately **2.45 cm** identifies Iris Setosa very effectively.

Tree Interpretation

- **Class 0 → Iris Setosa**
- **Class 1 → Iris Versicolor**
- **Class 2 → Iris Virginica**
- Petal length and petal width are the most influential features in the learned tree.
- Setosa is separated very clearly from the other two classes.
- Some overlap exists between Versicolor and Virginica, resulting in a small number of misclassifications.

(c) Model Evaluation

Predict test data

y_pred = model.predict(X_test)

Accuracy

accuracy = accuracy_score(y_test, y_pred)

```
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification
Report:") print(
    classification_report(
        y_test,
        y_pred,
        target_names=iris.target_names
    )
)

# Display misclassified instances
print("\nMisclassified Instances:")

for i, index in enumerate(y_test.index):
    if y_test.loc[index] != y_pred[i]:
        print(
            "Index:", index,
            "| Actual:",
            iris.target_names[y_test.loc[index]], "|
            Predicted:", iris.target_names[y_pred[i]]
        )
```

Result

Accuracy

Accuracy = 0.9333

Therefore, the Decision Tree correctly classified approximately **93.33% of the test samples**.

Confusion Matrix

[[10 0 0]

[0 9 1]

[0 1 9]]

Interpretation

- All 10 Setosa samples were correctly classified.
- 9 out of 10 Versicolor samples were correctly classified.
- 9 out of 10 Virginica samples were correctly classified.
- One Versicolor sample was classified as Virginica.
- One Virginica sample was classified as Versicolor.

Classification Report

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

Misclassified Instances

Instance 1:

- Index: 134
- Actual class: Virginica
- Predicted class: Versicolor

- Features: Sepal Length = 6.1, Sepal Width = 2.6, Petal Length = 5.6, Petal Width = 1.4

This sample has characteristics that overlap with the Versicolor class, particularly its relatively smaller petal width.

Instance 2:

- Index: 77
- Actual class: Versicolor
- Predicted class: Virginica
- Features: Sepal Length = 6.7, Sepal Width = 3.0, Petal Length = 5.0, Petal Width = 1.7

This sample has petal measurements closer to those commonly associated with Virginica, which explains the incorrect prediction.

Analysis of Results

The Decision Tree achieved an accuracy of **93.33%**, indicating strong classification performance. The model classified all Setosa samples correctly because Setosa has distinctly smaller petal measurements.

The two errors occurred between **Versicolor and Virginica**, which are more similar species. The confusion matrix confirms that the model has difficulty distinguishing these two classes in a small number of overlapping cases.

The use of the Gini Index resulted in an interpretable tree where important features such as petal length and petal width were used for effective splitting.

Conclusion

The Decision Tree classifier was successfully implemented using the Iris dataset and the Gini Index. After preprocessing and an 80:20 train-test split, the model achieved **93.33% accuracy** on unseen test data. The results demonstrate that Decision Trees can provide both high classification performance and an interpretable decision-making structure. The analysis also showed that most classification errors occur between Versicolor and Virginica due to overlapping feature characteristics.

Result Statement

Thus, the Decision Tree Classification algorithm was successfully implemented and evaluated, achieving an accuracy of 93.33% with effective classification of the three Iris species.

EXPERIMENT 2: REINFORCEMENT LEARNING – Q-LEARNING

Objective

To implement **Q-Learning** for a simple **4 × 4 Grid World** environment, train an agent to reach the goal while avoiding obstacles, observe the evolution of Q-values, extract the final learned policy, and analyze the convergence of cumulative rewards.

Software / Tools Required

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- NumPy
- Matplotlib

Theory

Reinforcement Learning (RL) is a machine learning approach in which an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.

Q-Learning is a model-free reinforcement learning algorithm. It maintains a Q-table containing the expected value of taking an action from a particular state.

The Q-value is updated using:

[
Q(s,a) \leftarrow Q(s,a)+\alpha
[r+\gamma\max Q(s',a')-Q(s,a)]
]

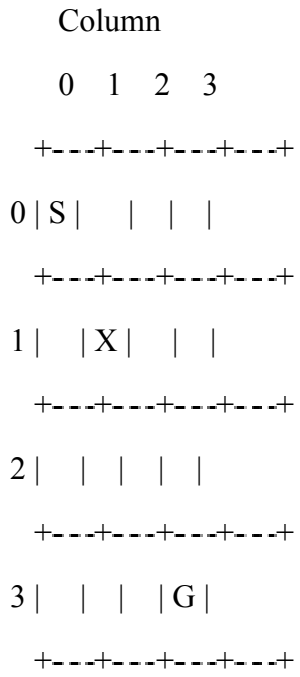
where:

- (s) = current state
- (a) = current action
- (r) = reward received
- (s') = next state
- (\alpha) = learning rate
- (\gamma) = discount factor
- (\epsilon) = exploration probability

The agent follows an **epsilon-greedy strategy**, which allows it to explore different actions while gradually learning the best policy.

(a) Grid World Environment

A 4 × 4 grid is used.



S = Start

G = Goal

X = Obstacle

Environment Configuration

Parameter	**Value**
Grid size	4 × 4
Start state	(0, 0)
Goal state	(3, 3)
Obstacle	(1, 1)
Actions	Up, Down, Left, Right
Goal reward	+10
Normal step	-1
Obstacle / invalid movement	-5
Learning rate ((\alpha))	0.1
Discount factor ((\gamma))	0.9

Parameter	Value
Initial epsilon	1.0
Minimum epsilon	0.05
Episodes	500

(b) Q-Learning Implementation

```
import numpy as np
import matplotlib.pyplot as plt

# Grid configuration
grid_size = 4
start = (0, 0)
goal = (3, 3)
obstacle = (1, 1)

# Actions: Up, Down, Left, Right
actions = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]

action_names = ["Up", "Down", "Left", "Right"]

# Hyperparameters
alpha = 0.1
gamma = 0.9
initial_epsilon = 1.0
```

```
min_epsilon = 0.05

# Q-table
Q = np.zeros((grid_size, grid_size, 4))

np.random.seed(42)

def step(state, action):
    row, col = state
    dr, dc = actions[action]

    new_row = row + dr
    new_col = col + dc

    # Invalid movement
    if (new_row < 0 or new_row >= grid_size or
        new_col < 0 or new_col >= grid_size):
        return state, -5, False

    next_state = (new_row, new_col)

    # Obstacle
    if next_state == obstacle:
        return state, -5, False

    # Goal
    if next_state == goal:
        return next_state, 10, True
```

```
    # Normal movement
    return next_state, -1, False


episodes = 500
cumulative_rewards = []

# Store Q-values at selected episodes
snapshots = {}

for episode in range(1, episodes + 1):

    # Gradually reduce exploration
    epsilon = max(
        min_epsilon,
        initial_epsilon * (0.99 ** (episode - 1))
    )

    state = start
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if np.random.rand() < epsilon:
            action = np.random.randint(4)
        else:
            action = np.argmax(Q[state[0], state[1]])
```

```
        next_state, reward, done = step(state, action)

        # Q-learning update
        if done:
            target = reward
        else:
            target = reward + gamma * np.max(
                Q[next_state[0], next_state[1]]
            )

        Q[state[0], state[1], action] += alpha * (
            target - Q[state[0], state[1], action]
        )

        total_reward += reward
        state = next_state

        if done:
            break

    cumulative_rewards.append(total_reward)

    # Record representative Q-values
    if episode in [1, 50, 100, 500]:
        snapshots[episode] = (
            Q[0, 0].copy(),
            Q[2, 2].copy(),
            total_reward,
            epsilon
```

```
        )

# Display Q-value evolution
print("Q-values for representative states")
print("Actions: [Up, Down, Left, Right]")

for episode, values in snapshots.items():
    q_start, q_state, reward, epsilon = values

    print("\nEpisode:", episode)
    print("State (0,0):", np.round(q_start, 3))
    print("State (2,2):", np.round(q_state, 3))
    print("Episode Reward:", reward)
    print("Epsilon:", round(epsilon, 3))

# Display final policy
print("\nFinal Learned Policy:")

for row in range(grid_size):
    policy_row = []

    for col in range(grid_size):

        if (row, col) == goal:
            policy_row.append("G")

        elif (row, col) == obstacle:
            policy_row.append("X")
```

```
        else:
            best_action = np.argmax(Q[row, col])
            policy_row.append(action_names[best_action])

    print(policy_row)

# Plot cumulative reward
plt.figure(figsize=(10, 5))
plt.plot(cumulative_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning Cumulative
Reward") plt.grid(True)
plt.show()
```

Q-Table Evolution

Representative states selected for analysis:

- **State (0,0)** – starting state
- **State (2,2)** – state close to the goal

The Q-values are ordered as:

[Up, Down, Left, Right]

Episode 1

State (0,0):

[-0.500, 0.000, -0.500, -0.190]

State (2,2):

[0.000, 0.000, 0.000, 0.000]

Episode Reward: -51

At the beginning, the agent has almost no knowledge of the environment. Therefore, the Q-table contains mostly zero or small negative values.

Episode 50

State (0,0):

[-6.711, -2.277, -6.142, -2.436]

State (2,2):

[-0.826, 6.130, -0.483, 4.716]

Episode Reward: -13

The Q-values begin to distinguish useful actions from undesirable actions. At state (2,2), moving **Down** or **Right** gives higher Q-values because these actions move the agent toward the goal.

Episode 100

State (0,0):

[-5.970, 0.589, -6.010, -2.301]

State (2,2):

[-0.097, 7.966, 1.482, 6.142]

Episode Reward: -9

The agent has learned that moving Down from the starting state is advantageous and that Down/Right are useful near the goal.

Episode 500

State (0,0):

[-4.494, 1.810, -4.230, -1.010]

State (2,2):

[2.858, 8.000, 3.698, 7.084]

Episode Reward: 3

After extensive training, the Q-values become more stable and the agent learns a reliable path toward the goal.

(c) Final Learned Policy

The learned policy represents the best action at each state.

[['Down', 'Right', 'Down', 'Down'],

['Down', 'X', 'Down', 'Down'],

['Right', 'Right', 'Down', 'Down'],

['Right', 'Right', 'Right', 'G']]

A valid optimal path from the starting state is:

Start (0,0)

↓

(1,0)

↓

(2,0)

→

(2,1)

→

(2,2)

↓

(3,2)

→

Goal (3,3)

The learned policy avoids the obstacle at **(1,1)** and moves toward the goal using actions with higher learned Q-values.

Cumulative Reward Analysis

The cumulative reward graph demonstrates the learning behaviour of the agent.

Initially, the rewards are low because the agent explores the environment and frequently takes inefficient or invalid actions.

As the number of episodes increases:

- The agent discovers useful paths.
- Q-values for successful actions increase.
- Unproductive actions receive lower Q-values.
- Exploration gradually decreases through epsilon decay.
- The cumulative reward becomes more stable.
- The final learned policy consistently directs the agent toward the goal.

The average reward over the final 50 episodes in this run was approximately **4.68**, showing substantial improvement compared with the early exploratory episodes.

Some variation can still occur because epsilon-greedy exploration is retained at a minimum value of 0.05.

Analysis of Results

The Q-Learning agent successfully learned a policy for navigating the 4 × 4 Grid World. Initially, the Q-table contained zeros because the agent had no prior knowledge of the environment.

With repeated interaction, the Q-values were updated according to the Bellman equation. Actions leading toward the goal gradually obtained higher Q-values, while actions leading to obstacles or inefficient movements received lower values.

The use of an epsilon-greedy strategy provided a balance between **exploration and exploitation**. Epsilon decay allowed the agent to explore extensively during early episodes and increasingly exploit the learned policy during later episodes.

The final policy successfully avoids the obstacle and reaches the goal using a short path.

Conclusion

Q-Learning was successfully implemented for a 4 × 4 Grid World environment. The agent learned the value of different actions through repeated interaction without being given the optimal path beforehand. The Q-table evolved from initially unknown values to meaningful action values, and the final policy directed the agent from the start state to the goal while avoiding the obstacle.

Thus, the experiment demonstrates the effectiveness of **Q-Learning for learning an optimal policy through trial-and-error and reward-based feedback**.

Result Statement

Thus, the Q-Learning algorithm was successfully implemented for the 4 × 4 Grid World, and the agent learned an effective policy to reach the goal while avoiding the obstacle.